UTM
UNIVERSITI TEKNOLOGI MALAYSIA

FALCULTY OF COMPUTING

SESSION: II 2025/2026

PROGRAM/CLASS:

SECJ3623

COURSE:

MOBILE APPLICATION PROGRAMMING

LECTURER:

AP. PROF.DR. MOHD SHAHIZAN OTHMAN

ASSESSMENT:

PROJECT PROPOSAL

NAME DETAILS:

NAME	REG. NO
THEVENDRAN A/L PAREMESWARAN	SX220343ECJHS04
MUHAMMAD FAIZAL BIN ASARAB ALI	SX210500ECJHS04
KIGENDRAN SIVA KUMAR	SX171065CSJS04

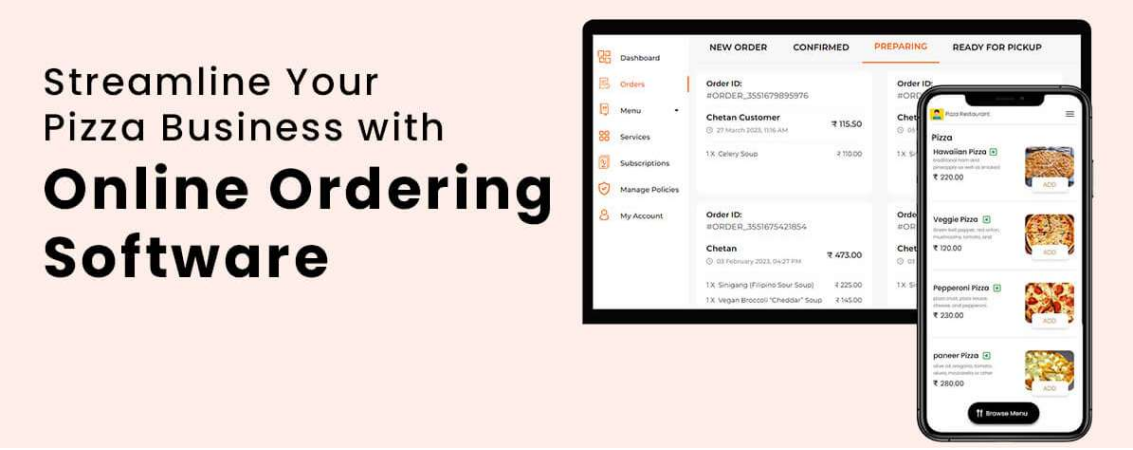
DATE:

05 OCT 2025

1.0 Executive Summary

The Pizza Ordering App System is a digital platform developed to streamline the process of ordering pizza through mobile and web applications. The system enables customers to conveniently browse the menu, select and customize pizzas, add side items, and place orders anytime and anywhere. It also supports multiple payment methods such as online banking, e-wallets, and cash on delivery. With features like real-time order tracking, delivery notifications, and user account management, the application enhances customer satisfaction by providing a fast, easy, and interactive ordering experience.

From the business perspective, the system assists restaurant administrators and staff in managing daily operations more efficiently. It provides functions to update menus, manage pricing, monitor incoming orders, track sales, and maintain customer records. By automating the ordering and payment processes, the system reduces manual errors, improves service speed, and optimizes workflow management. Overall, the Pizza Ordering App System serves as a scalable and user-friendly solution that supports business growth while meeting the evolving demands of digital food ordering services.



2.0 Development Approach and Workflow

❖ 2.1 Architecture Pattern

The application follows a simple Android architecture consisting of:

- Activities (User Interface)
- Java Classes (Logic handling)
- XML Layouts (UI design)
- Clear separation of concerns between UI, business logic, and data layers
- Reactive state management using streams
- Testability and maintainability
- Predictable state transitions

This separation ensures better readability and easier maintenance.The application follows a BLoC (Business Logic Component) architecture pattern.

❖ 2.2 Project Structure

The project is organized into:

PizzaOrderingApp Architecture

— Presentation Layer (UI)

│ ├— Screens (Views)

│ │ ├— Login / Sign Up Screen

│ │ ├— Menu / Pizza List Screen

│ │ ├— Pizza Customization Screen

│ │ ├— Cart Screen

│ │ ├— Payment Screen

│ │ └— Order Tracking Screen

│ │

```
│   └── Widgets (Reusable Components)
│       ├── Pizza Card Widget
│       ├── Cart Item Widget
│       ├── Quantity Selector
│       ├── Price Summary Bar
│       └── Payment Button

├── Business Logic Layer
│   ├── BLoCs (State Management)
│   │   ├── Authentication BLoC
│   │   ├── Menu BLoC
│   │   ├── Cart BLoC
│   │   ├── Order BLoC
│   │   └── Payment BLoC
│   │
│   └── Events & States
│       ├── Add to Cart Event
│       ├── Remove Item Event
│       ├── Place Order Event
│       ├── Payment Success State
│       └── Order Delivered State

├── Data Layer
```

```
│   ├── Repositories (Data Abstraction)
│   │   ├── User Repository
│   │   ├── Menu Repository
│   │   ├── Cart Repository
│   │   └── Order Repository
│   │
│   ├── Services (External APIs)
│   │   ├── Payment Gateway API
│   │   ├── Delivery / GPS Tracking API
│   │   └── Notification Service (SMS/Push)
│   │
│   └── Models (Data Structures)
│       ├── User Model
│       ├── Pizza Model
│       ├── Cart Model
│       ├── Order Model
│       └── Payment Model

└── Core / Utils
    ├── Constants
    │   ├── API Endpoints
    │   ├── App Colors
    │   └── Pricing Constants
```

└── Helpers

├── Validation Helpers

├── Date & Time Formatter

├── Price Calculator

└── Error Handle

❖ 2.3 Development Workflow

1. **Requirement Analysis**
   Gather system requirements such as user registration, menu browsing, pizza customization, cart, payment, and delivery tracking.
2. **System Design**
   Design system architecture, database schema, and UML diagrams (use case, class, sequence).
3. **UI/UX Design**
   Create user-friendly interfaces for menu display, cart, checkout, and order tracking screens.
4. **Frontend Development**
   Develop mobile/web interfaces using technologies such as Flutter, Android, or React.
5. **Backend Development**
   Build server-side logic to handle orders, authentication, payments, and notifications.
6. **Database Development**
   Design and implement database tables for users, pizzas, orders, payments, and delivery details.

7. **API Integration**

   Integrate REST APIs for payment gateways, maps/GPS tracking, and push notifications.

8. **State Management Implementation**

   Apply state management (e.g., BLoC, Provider, MVVM) to manage cart, menu, and order states.

9. **Testing & Debugging**

   Conduct unit testing, integration testing, and user acceptance testing to ensure system reliability.

10. **Deployment & Maintenance**

    Deploy the app to cloud/server platforms and perform updates, bug fixes, and performance monitoring.

3.0 Software, Tools and Frameworks

❖ 3.1 Core Technologies

Technology	Version	Purpose
Flutter	Latest Stable	Cross-platform mobile framework
Dart	3.0+	Programming language
Firebase	Latest	Backend services

❖ 3.2 Key Dependencies

- **Flutter SDK**

  Main framework for cross-platform mobile app development.

- **flutter_bloc: ^8.0.0**

  Implements BLoC pattern for state management (cart, menu, orders).

- **firebase_core: ^2.0.0**

  Initializes Firebase services within the app.

- **firebase_auth: ^4.0.0**

  Provides user authentication and account management.

- **cloud_firestore: ^4.0.0**

  Stores and retrieves app data (users, menu, orders) using a NoSQL database.

- **firebase_storage: ^11.0.0**

  Stores pizza images, profile pictures, and other media files.

- **firebase_messaging: ^14.0.0**

  Enables push notifications for order updates and promotions.

- **flutter_local_notifications: ^15.0.0**

  Displays local notifications on the user's device for real-time alerts.

- **pdf**

  Generates PDF invoices, receipts, and reports for orders.

- **image_picker**

  Allows uploading and selecting images from camera or gallery.

- **intl**

  Supports internationalization, date/time formatting, and currency display.

❖ 3.3 Development Tools

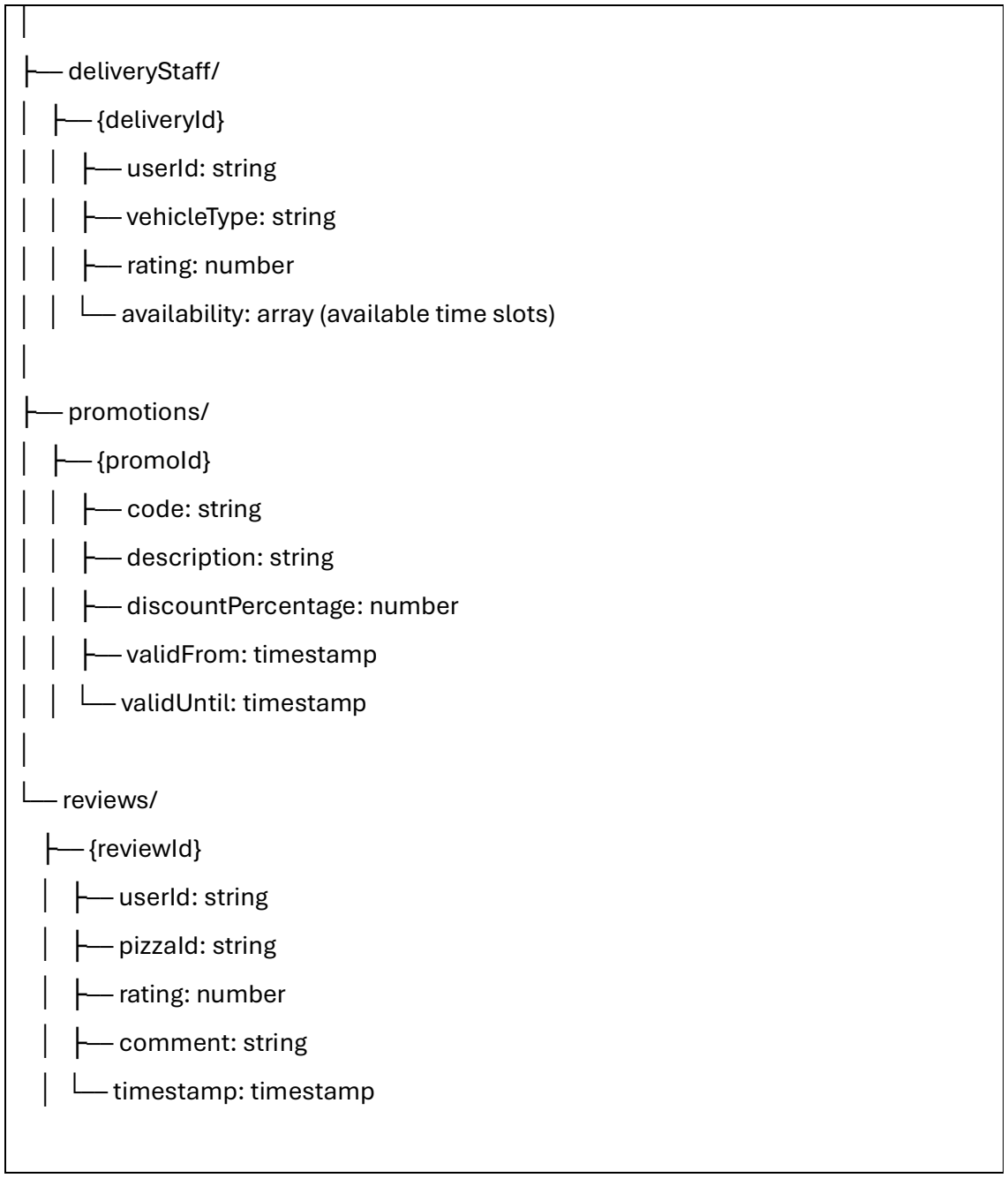
Tool	Purpose
Android Studio	Development
Java	Programming
XML	UI Design
Android SDK	Build tools
Android Device	Deployment

4.0 Database and Data Handling Design

❖ 4.1 Firebase Firestore Structure

The application uses Cloud Firestore with the following collection structure

```
firestore
├── users/
│   ├── {userId}
│   │   ├── email: string
│   │   ├── name: string
│   │   ├── phoneNumber: string
│   │   ├── address: string
│   │   ├── profileImageUrl: string
│   │   └── role: string (customer/admin/delivery)
│
├── pizzas/
│   ├── {pizzaId}
│   │   ├── name: string
│   │   ├── description: string
│   │   ├── price: number
│   │   ├── imageUrl: string
│   │   └── category: string (veg/non-veg/special)
│
├── orders/
│   ├── {orderId}
│   │   ├── userId: string
│   │   ├── pizzaItems: array (list of pizzaId + quantity + customization)
│   │   ├── totalPrice: number
│   │   ├── orderDate: timestamp
│   │   ├── status: string (pending/processing/delivered/cancelled)
│   │   └── deliveryAddress: string
```


deliveryStaff/
{deliveryId}
userId: string
vehicleType: string
rating: number
availability: array (available time slots)
promotions/
{promoId}
code: string
description: string
discountPercentage: number
validFrom: timestamp
validUntil: timestamp
reviews/
{reviewId}
userId: string
pizzaId: string
rating: number
comment: string
timestamp: timestamp

❖ 4.2 Data Models

Example AppUser Model:

```
enum UserRole { customer, admin, delivery }


class AppUser {
 final String id;
 final String email;
 final String name;
 final String? phoneNumber;
 final String? address;
 final String? profileImageUrl;
 final UserRole role;

 AppUser({
  required this.id,
  required this.email,
  required this.name,
  this.phoneNumber,
  this.address,
  this.profileImageUrl,
  required this.role,
 });

 // Convert AppUser object to a map for Firestore
 Map<String, dynamic> toMap() {
  return {
   'id': id,
   'email': email,
   'name': name,
   'phoneNumber': phoneNumber,
```

```
    'address': address,
    'profileImageUrl': profileImageUrl,
    'role': role.toString(), // Stored as string
   };
  }

  // Create AppUser object from Firestore map
  factory AppUser.fromMap(Map<String, dynamic> map) {
   return AppUser(
    id: map['id'],
    email: map['email'],
    name: map['name'],
    phoneNumber: map['phoneNumber'],
    address: map['address'],
    profileImageUrl: map['profileImageUrl'],
    role: UserRole.values.firstWhere(
     (e) => e.toString() == map['role'],
    ),
   );
  }
}
```

5.0 Implementation CRUD Operation

❖ 5.1 Create Operation

BLoC Event Handler for Adding a Pizza Order :

```
on<AddOrder>((event, emit) async {
 // Set state to loading while processing the order
 emit(state.copyWith(status: OrderStatus.loading));

 try {
  // Optional: upload image if the order has a custom pizza image
  String? imageUrl;
  if (event.order.imageFile != null) {
   imageUrl = await _storageService.uploadPizzaImage(
    event.order.id,
    event.order.imageFile!,
   );
  }

  // Copy the order with the uploaded image URL
  final order = event.order.copyWith(imageUrl: imageUrl);

  // Add order to Firestore / repository
  await _orderRepository.addOrder(order);

  // Update state with the new order list
  emit(state.copyWith(
   status: OrderStatus.success,
   orders: [...state.orders, order],
```

```
  ));
 } catch (e) {
  // If any error occurs, emit error state
  emit(state.copyWith(
   status: OrderStatus.error,
   errorMessage: e.toString(),
  ));
 }
});
```

❖ 5.2 Read Operation

Real-time Data Subscription

```
on<LoadOrders>((event, emit) async {
 // Set state to loading while fetching orders
 emit(state.copyWith(status: OrderStatus.loading));

 // Listen to the orders stream from repository
 await emit.forEach<List<Order>>(
  _orderRepository.getOrdersByUser(event.userId),
  onData: (orders) => state.copyWith(
   status: OrderStatus.success,
   orders: orders,
  ),
  onError: (error, stackTrace) => state.copyWith(
   status: OrderStatus.error,
   errorMessage: error.toString(),
  ),
 );
});
```

❖ 5.3 Update Operation

BLoC Handler for Updating a Pizza Order:

```
on<UpdateOrder>((event, emit) async {
 // Set state to loading while updating the order
 emit(state.copyWith(status: OrderStatus.loading));

 try {
  // Get the existing image URL (if any)
  String? imageUrl = event.order.imageUrl;

  // If a new image file is provided, upload it and delete the old one
  if (event.newImageFile != null) {
   if (imageUrl != null) {
    await _storageService.deleteFile(imageUrl);
   }
   imageUrl = await _storageService.uploadPizzaImage(
    event.order.id,
    event.newImageFile!,
   );
  }

  // Create a copy of the order with the new image URL
  final updatedOrder = event.order.copyWith(imageUrl: imageUrl);

  // Update order in the repository / Firestore
  await _orderRepository.updateOrder(updatedOrder);

  // Update the orders list in state
  final updatedOrders = state.orders.map((order) =>
    order.id == updatedOrder.id ? updatedOrder : order
```

```
    ).toList();

    // Emit success state with updated orders
    emit(state.copyWith(
      status: OrderStatus.success,
      orders: updatedOrders,
    ));
  } catch (e) {
    // Emit error state if something goes wrong
    emit(state.copyWith(
      status: OrderStatus.error,
      errorMessage: e.toString(),
    ));
  }
});
```

❖ 5.4 Delete Operation

BLoC Handler for Deleting a Pizza:

```
on<DeleteOrder>((event, emit) async {
  // Set state to loading while deleting the order
  emit(state.copyWith(status: OrderStatus.loading));

  try {
    // Delete order image from storage if exists
    if (event.order.imageUrl != null) {
      await _storageService.deleteFile(event.order.imageUrl!);
    }

    // Delete order from repository / Firestore
    await _orderRepository.deleteOrder(event.order.id);
```

```
  // Remove deleted order from state list
  final updatedOrders =
      state.orders.where((order) => order.id != event.order.id).toList();

  // Emit success state with updated orders
  emit(state.copyWith(
    status: OrderStatus.success,
    orders: updatedOrders,
  ));
 } catch (e) {
  // Emit error state if deletion fails
  emit(state.copyWith(
    status: OrderStatus.error,
    errorMessage: e.toString(),
  ));
 }
});
```

6.0 Important Source Code Snippets

- 6.1 Main Application Initialization

From main.dart - Application register point

```
void main() async {
 // Ensure Flutter binding is initialized
 WidgetsFlutterBinding.ensureInitialized();

 // Initialize Firebase
 await Firebase.initializeApp(
  options: DefaultFirebaseOptions.currentPlatform,
 );

 // Set up BLoC observer
 Bloc.observer = AppBlocObserver();

 // Initialize services
 final notificationService = NotificationService(
  FirebaseMessaging.instance,
  FlutterLocalNotificationsPlugin(),
 );

 final authService = FirebaseAuthService(FirebaseAuth.instance);
 final firestoreService = FirestoreService(FirebaseFirestore.instance);
 final storageService = StorageService(FirebaseStorage.instance);
 final pdfService = PdfService();

 // Initialize repositories
 final authRepository = AuthRepository(
  authService: authService,
```

```
    firestoreService: firestoreService,
  );

  final userRepository = UserRepository(firestoreService: firestoreService);
  final pizzaRepository = PizzaRepository(firestoreService: firestoreService);
  final orderRepository = OrderRepository(firestoreService: firestoreService);
  final deliveryRepository = DeliveryRepository(firestoreService: firestoreService);

  // Run the app with all dependencies
  runApp(PizzaOrderingApp(
    authRepository: authRepository,
    userRepository: userRepository,
    pizzaRepository: pizzaRepository,
    orderRepository: orderRepository,
    deliveryRepository: deliveryRepository,
    storageService: storageService,
    notificationService: notificationService,
    pdfService: pdfService,
  ));
}
```

Explanation: Ensures Flutter bindings are initialized before Firebase, initializes all Firebase services and custom services, and creates repository instances with dependency injection for clean separation of concerns

❖ 6.2 Dependency Injection Setup

Widget tree dependency configuration:

```
@override
Widget build(BuildContext context) {
 return MultiRepositoryProvider(
  providers: [
   // Repositories made available throughout the widget tree
   RepositoryProvider.value(value: widget.authRepository),
   RepositoryProvider.value(value: widget.userRepository),
   RepositoryProvider.value(value: widget.pizzaRepository),
   RepositoryProvider.value(value: widget.orderRepository),
   RepositoryProvider.value(value: widget.deliveryRepository),
   RepositoryProvider.value(value: widget.storageService),
   RepositoryProvider.value(value: widget.notificationService),
   RepositoryProvider.value(value: widget.pdfService),
  ],
  child: MultiBlocProvider(
   providers: [
    // BLoCs with required dependencies
    BlocProvider(
     create: (_) => AuthBloc(widget.authRepository)
      ..add(const AuthStatusRequested()),
    ),
    BlocProvider(
     create: (_) => UserBloc(userRepository: widget.userRepository),
    ),
    BlocProvider(
     create: (_) => PizzaBloc(
      pizzaRepository: widget.pizzaRepository,
      storageService: widget.storageService,
```

```
        ),
      ),
      BlocProvider(
        create: (_) => OrderBloc(
          orderRepository: widget.orderRepository,
          storageService: widget.storageService,
          notificationService: widget.notificationService,
        ),
      ),
      BlocProvider(
        create: (_) => DeliveryBloc(
          deliveryRepository: widget.deliveryRepository,
          notificationService: widget.notificationService,
        ),
      ),
    ],
    child: MaterialApp(
      title: 'Pizza Ordering App',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        primarySwatch: Colors.red,
      ),
      home: const SplashScreen(), // Replace with your initial screen
    ),
  ),
 );
}
```

Explanation: MultiRepositoryProvider makes repositories available throughout the widget tree. MultiBlocProvider creates and provides BLoC instances with their required dependencies. Initial events are dispatched on creation.

❖ 6.3 Authentication Flow with BLoC Listener

```
@override
Widget build(BuildContext context) {
 return MultiRepositoryProvider(
  providers: [
   // Repositories made available throughout the widget tree
   RepositoryProvider.value(value: widget.authRepository),
   RepositoryProvider.value(value: widget.userRepository),
   RepositoryProvider.value(value: widget.pizzaRepository),
   RepositoryProvider.value(value: widget.orderRepository),
   RepositoryProvider.value(value: widget.deliveryRepository),
   RepositoryProvider.value(value: widget.storageService),
   RepositoryProvider.value(value: widget.notificationService),
   RepositoryProvider.value(value: widget.pdfService),
  ],
  child: MultiBlocProvider(
   providers: [
    // BLoCs with required dependencies
    BlocProvider(
     create: (_) => AuthBloc(widget.authRepository)
      ..add(const AuthStatusRequested()),
    ),
    BlocProvider(
     create: (_) => UserBloc(userRepository: widget.userRepository),
    ),
    BlocProvider(
     create: (_) => PizzaBloc(
      pizzaRepository: widget.pizzaRepository,
      storageService: widget.storageService,
     ),
    ),
```

```
      BlocProvider(
        create: (_) => OrderBloc(
          orderRepository: widget.orderRepository,
          storageService: widget.storageService,
          notificationService: widget.notificationService,
        ),
      ),
      BlocProvider(
        create: (_) => DeliveryBloc(
          deliveryRepository: widget.deliveryRepository,
          notificationService: widget.notificationService,
        ),
      ),
    ],
    child: MaterialApp(
      title: 'Pizza Ordering App',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        primarySwatch: Colors.red,
      ),
      home: const SplashScreen(), // Replace with your initial screen
    ),
  ),
 );
}
```

Explanation: The BlocListener is used to monitor authentication state changes without rebuilding the UI. It listens to the AuthBloc and reacts whenever the authentication status changes, ensuring that the app responds appropriately to login or logout events. The listenWhen condition prevents unnecessary execution of the listener when the status remains the same, improving efficiency. For navigation, the _navigatorKey allows the app

to perform navigation outside the standard widget context, while pushNamedAndRemoveUntil clears the navigation stack when logging out, ensuring users cannot return to previous screens. The routing has been updated specifically for the Pizza Ordering App, including HomeScreen, MenuScreen, CartScreen, OrdersScreen, and ProfileScreen, so that the login flow now directs users to the correct screens according to their authentication state.

❖ 6.4 Firestore Service Implementation

```
class FirestoreService {
 final FirebaseFirestore _firestore;
 FirestoreService(this._firestore);
 // Get a collection reference
 CollectionReference collection(String path) {
  return _firestore.collection(path);
 }
 // Get a single document by ID
 Future<DocumentSnapshot> getDocument(String path, String id) async {
  return await _firestore.collection(path).doc(id).get();
 }
 // Get a stream of documents with optional filtering and ordering
 Stream<QuerySnapshot> getCollectionStream(
  String path, {
  List<WhereCondition>? where,
  String? orderBy,
 }) {
  Query query = _firestore.collection(path);

  // Apply where conditions
  if (where != null) {
   for (var condition in where) {
```

```
      query = query.where(condition.field, isEqualTo: condition.value);
    }
   }
   // Apply ordering
   if (orderBy != null) {
    query = query.orderBy(orderBy);
   }

   return query.snapshots();
  }
}
// Optional helper class for queries
class WhereCondition {
 final String field;
 final dynamic value;
 WhereCondition({required this.field, required this.value});
}
```

Explanation: The Firestore Service wraps Firebase Firestore to simplify testing and mocking. It provides generic CRUD operations for collections and documents, supports real-time streams for reactive UI updates, and allows flexible query composition using where clauses and ordering. In the context of the Pizza Ordering App, it can be used to manage user profiles under the users/ collection, fetch menu items from pizzas/, stream orders for customers or delivery staff in real time from orders/, query available delivery personnel from deliveryStaff/, and retrieve customer feedback from the reviews/ collection.

❖ 6.5 Storage Service for Image Handling

```
class StorageService {
```

```
  final FirebaseStorage _storage
  StorageService(this._storage);
  // Upload pizza or order image and return the download URL
  Future<String> uploadPizzaImage(String itemId, File imageFile) async {
   try {
    final ref = _storage
        .ref()
        .child('pizzas/$itemId/${DateTime.now().millisecondsSinceEpoch}');
    final uploadTask = ref.putFile(imageFile);
    final snapshot = await uploadTask.whenComplete(() {});
    final downloadUrl = await snapshot.ref.getDownloadURL();
    return downloadUrl;
   } catch (e) {
    throw Exception('Failed to upload image: $e');
   }
  }
  // Delete a file from Firebase Storage
  Future<void> deleteFile(String url) async {
   try {
    final ref = _storage.refFromURL(url);
    await ref.delete();
   } catch (e) {
    throw Exception('Failed to delete file: $e');
   }
  }
}
```

The StorageService encapsulates all Firebase Storage operations for the Pizza Ordering App. It generates unique file paths for each pizza or order image, uploads the files, and returns the download URLs that can be stored in Firestore. The service also handles cleanup by deleting old images when pizzas or orders are updated, ensuring that storage

remains organized and up to date. In practice, it can be used to upload pizza images for the menu, upload custom images for orders such as special pizza requests, and delete images when pizzas are removed or updated.

7.0 Technical Challenges and Solutions

❖ 7.1 Asynchronous Initialization

Problem: Firebase services, such as authentication, Firestore, and notifications, require asynchronous initialization, which can delay app startup.

Solution: Critical services like Firebase are initialized synchronously in main(), while non-critical services, such as push notifications, are initialized asynchronously in the widget's initState().

Outcome: Faster app startup with smoother user experience and reduced waiting time for users.

❖ 7.2 State Management Complexity

Problem: Managing multiple interconnected states across screens (menu, cart, orders, delivery) can become complex.

Solution: Implemented the BLoC pattern with domain-specific BLoCs: AuthBloc, UserBloc, PizzaBloc, OrderBloc, and DeliveryBloc.

Outcome: Predictable state transitions, easier debugging, and fully testable business logic for each domain.

❖ 7.3 Real-time Data Synchronization

Problem: Keeping the UI updated in real-time when Firestore data changes, such as orders, menu updates, or delivery status.

Solution: Used Firestore streams instead of futures, with BLoCs subscribing to these streams using emit.forEach().

Outcome: Automatic UI updates without manual refresh, ensuring orders, menu items, and delivery statuses are always current.

❖ 7.4 Navigation After Logout

Problem: Users could navigate back to authenticated screens after logging out.

Solution: Applied pushNamedAndRemoveUntil() to clear the entire navigation stack when logging out.

Outcome: Improved security and consistent authentication flow, preventing unauthorized access to previous screens.

❖ 7.5 Image Upload and Management

Problem: Handling pizza images or order images efficiently, including uploads, storage, and cleanup.

Solution: Encapsulated image operations in StorageService, deleting old images before uploading new ones to generate unique, up-to-date URLs.

Outcome: Optimized storage usage, consistent image URLs, and robust error handling.

❖ 7.6 Role-Based Access Control

Problem: Different user types (customer, admin, delivery) require access to different features and screens.

Solution: Used a UserRole enum and implemented conditional routing and UI rendering based on user roles.

Outcome: Secure access control and customized user experience for each role, such as customers seeing menu and orders, admins managing menu items, and delivery staff updating order status.

8.0 Security Consideration

❖ 8.1 Authentication Security

1. Firebase Authentication is used to handle password hashing and secure token management, ensuring that user credentials are protected.
2. Email verification is enforced for new accounts to validate the user's identity.
3. Password reset functionality is implemented using secure tokens, allowing users to safely recover their accounts.

❖ 8.2 Data Access Rules

Firestore security rules are configured to enforce **user-specific access** for sensitive data. Example rules adapted for the Pizza Ordering App:

```
rules_version = '2';
service cloud.firestore {
 match /databases/{database}/documents {

  // Users collection: only authenticated users can read; users can only write their own data
  match /users/{userId} {
   allow read: if request.auth != null;
   allow write: if request.auth.uid == userId;
  }

  // Pizzas collection: anyone can read menu; only admins can write
  match /pizzas/{pizzaId} {
   allow read: if request.auth != null;
   allow write: if request.auth != null && request.auth.token.role == 'admin';
  }
```

```
    // Orders collection: users can read their own orders; delivery staff can read assigned orders
    match /orders/{orderId}{
      allow read: if request.auth != null &&
            (resource.data.userId == request.auth.uid ||
             resource.data.deliveryId == request.auth.uid);
      allow write: if request.auth != null && resource.data.userId == request.auth.uid;
    }

    // Delivery staff collection: only admins can write; delivery staff can read their own profile
    match /deliveryStaff/{deliveryId}{
      allow read: if request.auth != null && request.auth.uid == resource.data.userId;
      allow write: if request.auth != null && request.auth.token.role == 'admin';
    }
  }
}
```

❖ 8.3 Storage Security

1. Firebase Storage rules enforce **authenticated access only**, ensuring that only logged-in users can upload or retrieve files.
2. File paths are structured to be **user-specific**, preventing unauthorized access to other users' pizza images, order attachments, or profile photos.

9.0 Performance Optimizations

❖ 9.1 Lazy Loading

1. BLoCs such as PizzaBloc, OrderBloc, and DeliveryBloc are created **only when needed**, reducing memory usage and startup time.
2. Pizza images and order-related images are loaded **on demand**, with caching applied to avoid repeated network calls and improve scrolling performance in menus and order lists.

❖ 9.2 Real-time Data Optimization

1. Firestore queries are optimized using **indexes** for faster retrieval of pizzas, orders, and delivery staff.
2. Collection queries are **limited and filtered with where clauses** to reduce unnecessary data transfer. For example, fetching only orders for the logged-in user or delivery staff.
3. **Pagination** is recommended for large lists, such as a long menu of pizzas or historical orders, to improve performance and reduce memory overhead.

❖ 9.3 State Management Efficiency

1. The listenWhen and buildWhen conditions in BLoCs prevent **unnecessary widget rebuilds**, improving UI responsiveness.
2. emit.forEach efficiently handles Firestore stream subscriptions, keeping the UI synchronized with real-time data while minimizing performance overhead.
3. Immutable state using copyWith ensures **predictable updates**, making state changes efficient, easy to debug, and testable across the Pizza Ordering App.

10.0 Testing Strategy

❖ 10.1 Unit Testing

1. Business logic in BLoCs such as PizzaBloc, OrderBloc, AuthBloc, and DeliveryBloc is tested in isolation.
2. Mock repositories and services (Firestore, Storage, NotificationService) are used to simulate backend responses.
3. Tests focus on **state transitions** and **event handling**, ensuring that BLoCs react correctly to user actions like adding an order or updating a pizza.

```
void main() {
 group('OrderBloc', () {
  late OrderRepository mockRepository;
  late OrderBloc orderBloc;

  setUp(() {
   mockRepository = MockOrderRepository();
   orderBloc = OrderBloc(orderRepository: mockRepository);
  });

  test('emits success state when orders are loaded', () async {
   when(() => mockRepository.getOrdersByUser(any()))
     .thenAnswer((_) => Stream.value([mockOrder]));

   expectLater(
    orderBloc.stream,
    emitsInOrder([
     isA<OrderState>().having((s) => s.status, 'status', OrderStatus.loading),
     isA<OrderState>().having((s) => s.status, 'status', OrderStatus.success),
    ]),
```

```
    );

    orderBloc.add(LoadOrders(userId: 'test-user'));
  });
 });
}
```

❖ 10.2 Widget Testing

1. UI components are tested in isolation with mocked BLoC states.
2. User interactions such as adding items to cart, placing orders, or navigating menus are verified.

❖ 10.3 Integration Testing

1. Complete user flows are tested, including signup/login, browsing pizzas, placing orders, and delivery updates.
2. Firebase integration is validated for authentication, Firestore, Storage, and notifications.
3. Navigation flows between screens (Menu, Cart, Orders, Profile) are verified for correctness.

11.0 Future Enhancements

❖ 11.1 Recommended Technical Improvements

1. **Pagination:** Implement pagination for large pizza menus or order histories.
2. **Offline Support:** Enhance offline capabilities with a local database (Hive/Drift).
3. **Error Tracking:** Integrate Crashlytics for production error monitoring.
4. **Analytics:** Add Firebase Analytics to track user behavior and order trends.
5. **Automated Testing:** Increase test coverage to 80%+ across BLoCs and widgets.
6. **CI/CD:** Set up automated build and deployment pipelines.
7. **Performance Monitoring:** Integrate Firebase Performance Monitoring to track app efficiency.
8. **Search Functionality:** Implement Algolia or Elasticsearch for quick pizza/menu searches.
9. **Payment Integration:** Add Stripe or PayPal for order payments.
10. **Chat Feature:** Enable real-time messaging between customers and delivery staff.

12.0 Conclusion

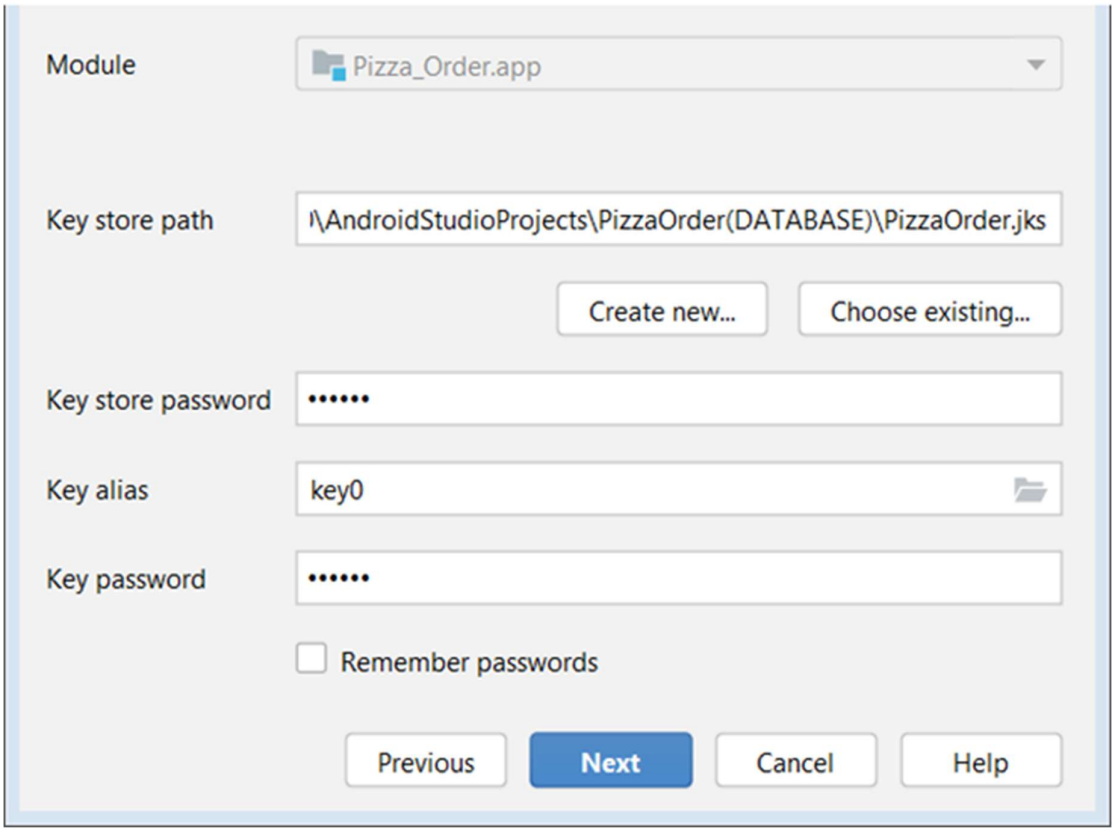
The Pizza Ordering App demonstrates a well-architected Flutter application following industry best practices. The application successfully integrates multiple Firebase services—Authentication, Firestore, Storage, and Messaging—with a robust state management solution using BLoC, providing a solid foundation for a production-ready platform.

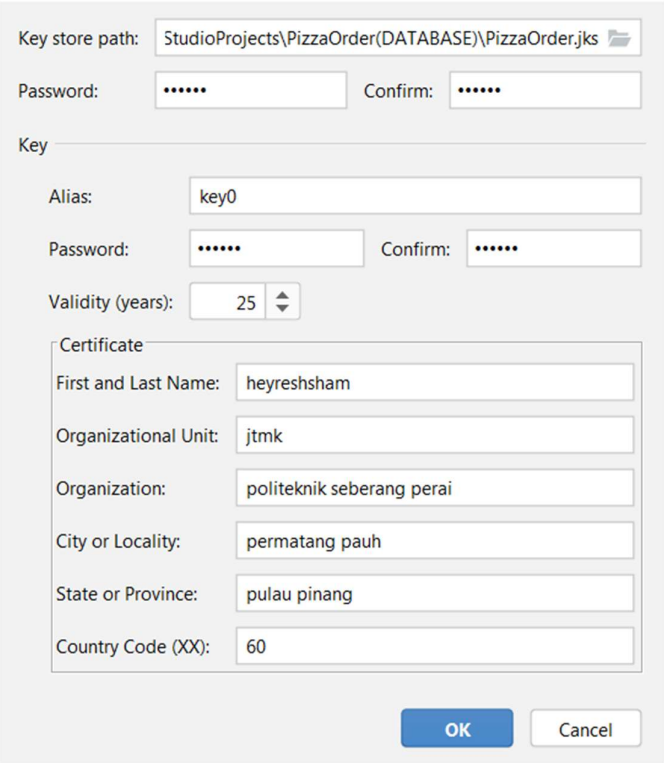
Key Strengths:

- **Clean Architecture:** Clear separation of concerns with the BLoC pattern.
- **Scalability:** Modular design allows easy addition of new features like promotions or menu categories.
- **Maintainability:** Consistent code structure and design patterns enable easy updates and debugging.
- **Real-time Capabilities:** Leverages Firebase streams to sync orders, menu updates, and delivery status in real-time.
- **User Experience:** Responsive UI with proper loading indicators and error handling.
- **Security:** Firebase Authentication, Firestore rules, and Storage rules ensure data protection and role-based access control for customers, admins, and delivery staff.

5.3 Creating New Keystore

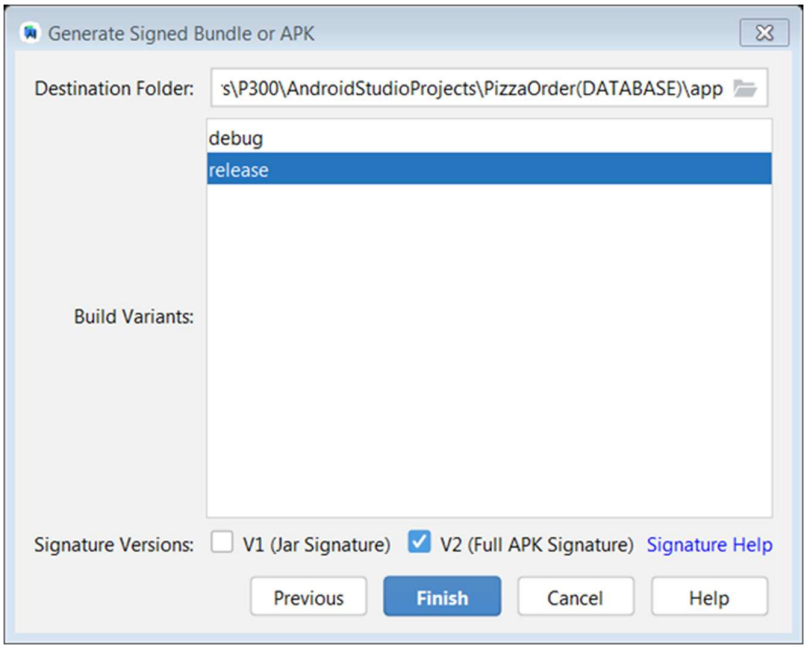
A keystore was created with alias, password, and validity.





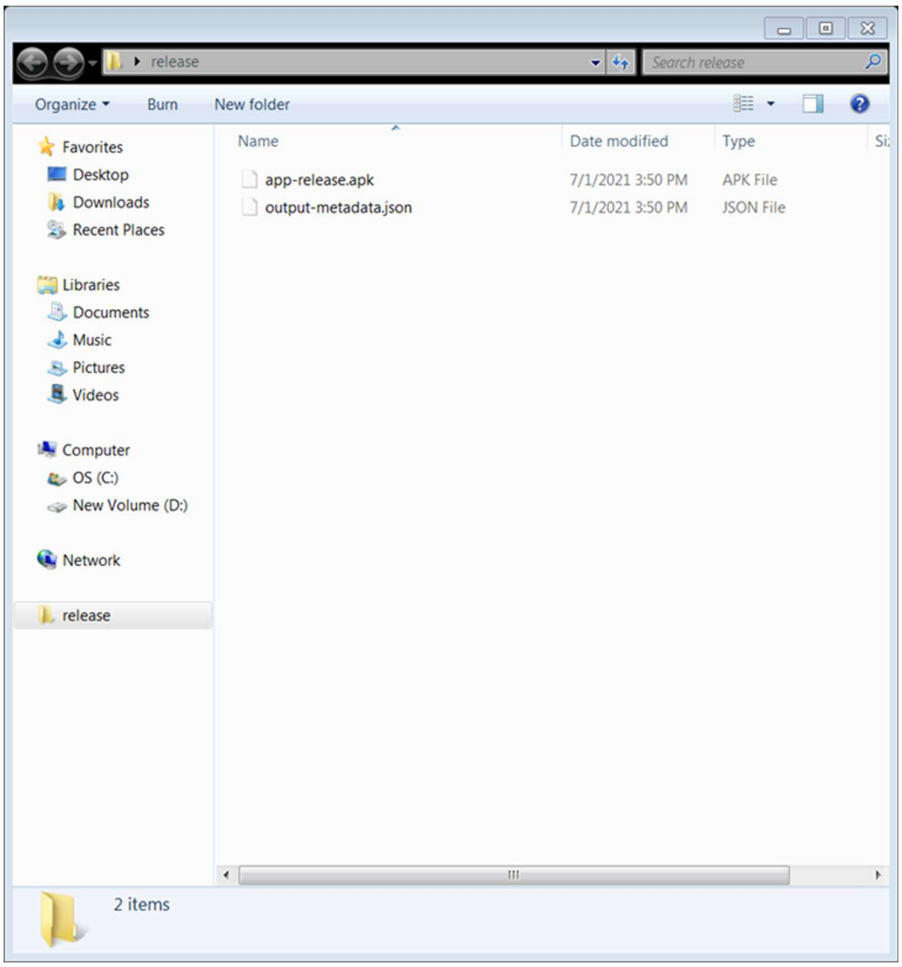
5.4 Selecting Release Build

Release build selected with V1 and V2 signatures.



5.5 APK Generated Successfully

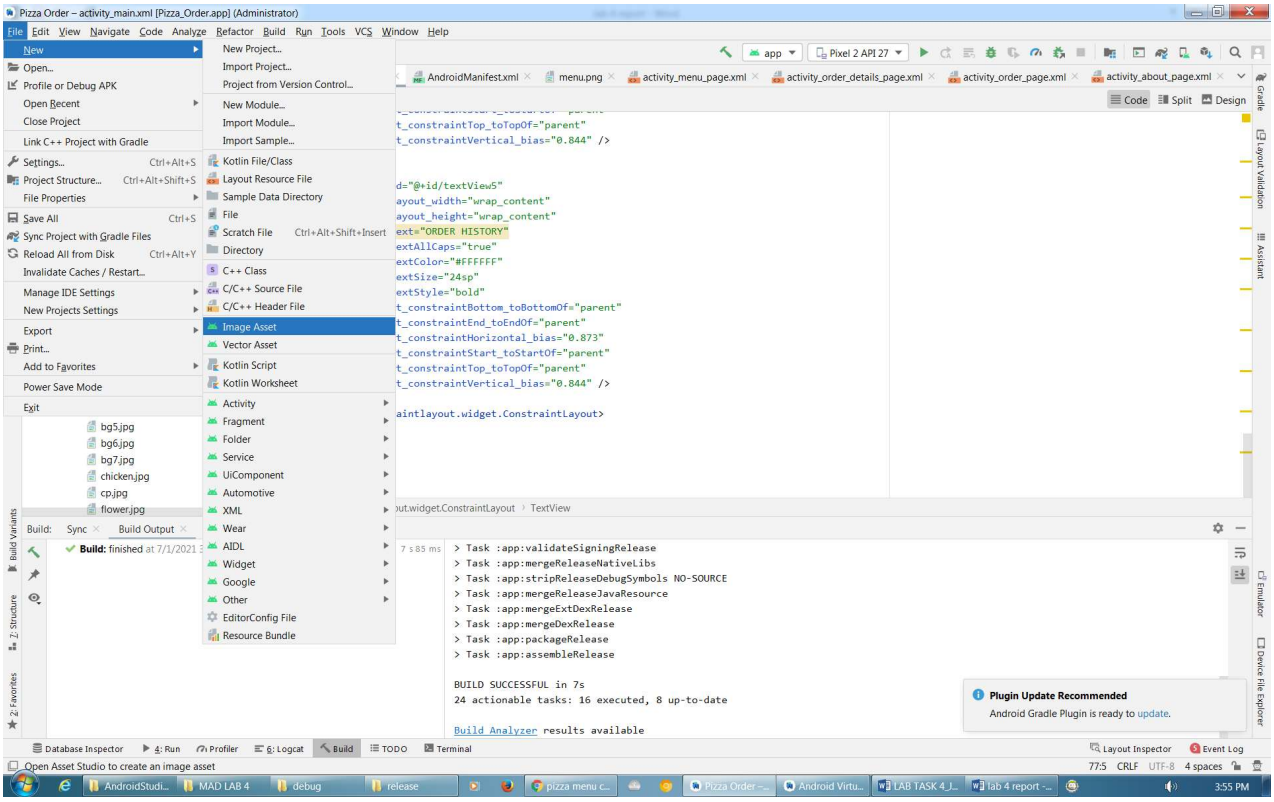
Signed APK successfully generated.



6.0 Customizing Application Icon

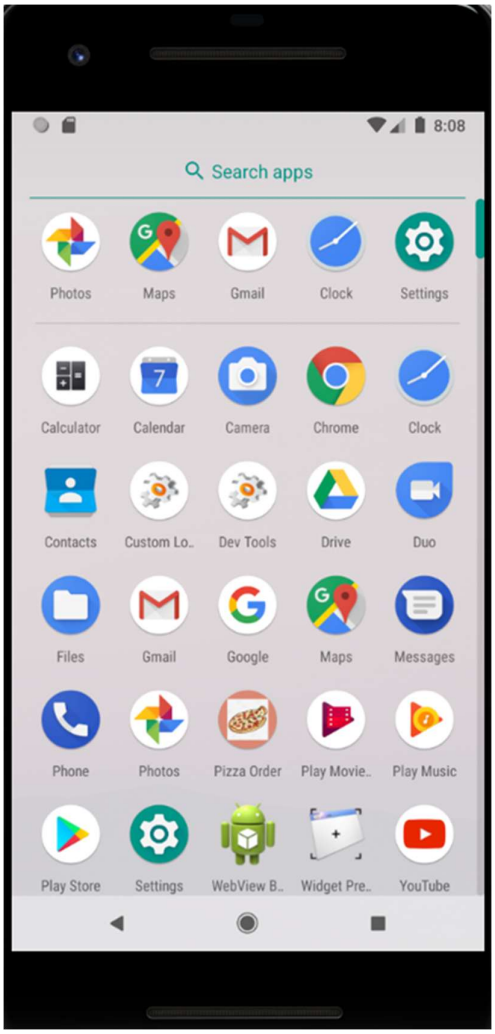
6.1 Image Asset Tool

res → mipmap → New → Image Asset



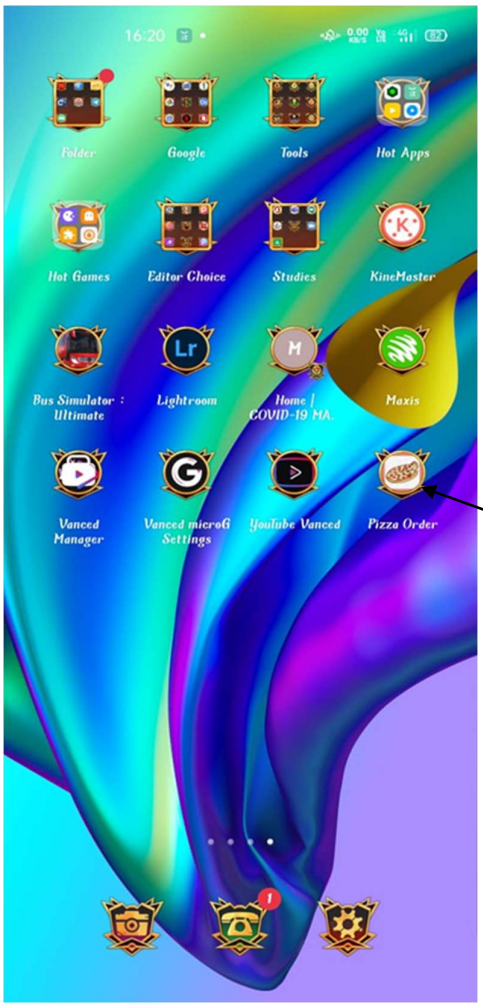
6.2 Icon Preview and Generation

All launcher icon sizes generated.

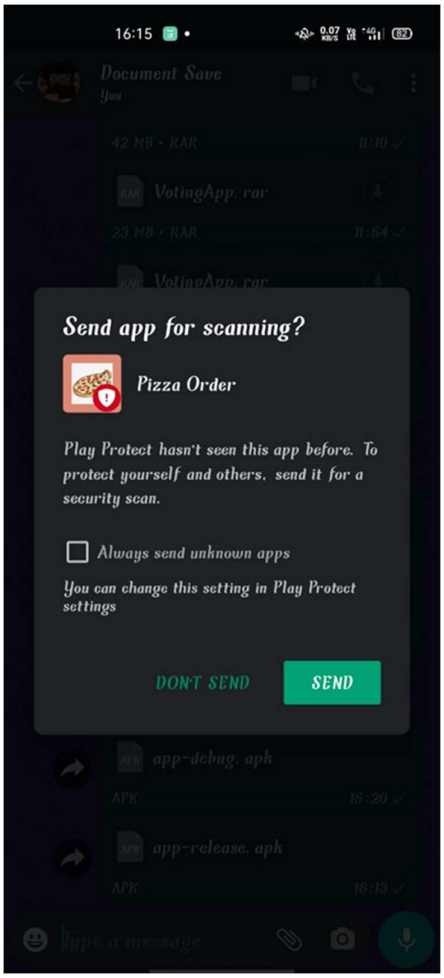


7.0 Installing Application on Mobile Device

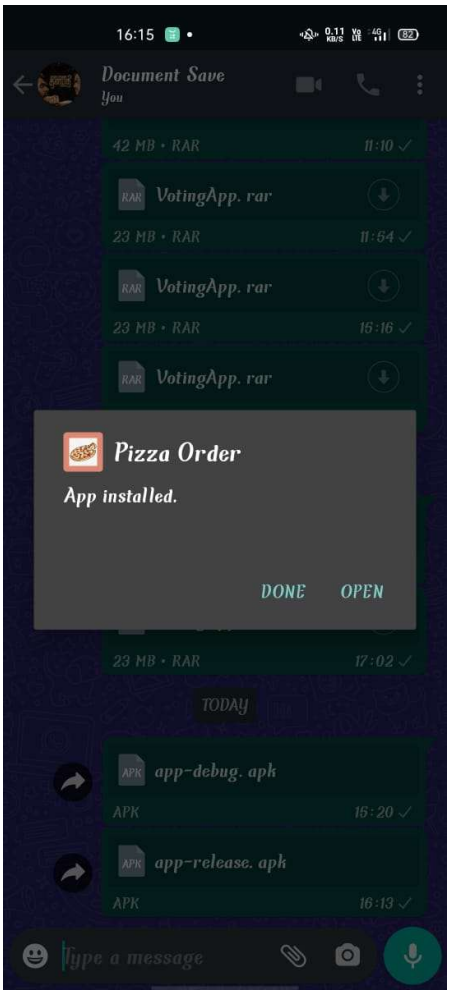
7.1 Transferring APK to Phone



7.2 Allow Installation from Unknown Sources



7.3 Application Installed and Running


16:15
Document Save
You
42 MB • RAR 11:10
VotingApp. rar
23 MB • RAR 11:64
VotingApp. rar
23 MB • RAR 15:16
VotingApp. rar
Pizza Order
App installed.
DONE
OPEN
23 MB • RAR 17:02
TODAY
app-debug. apk
APK 15:20
app-release. apk
APK 16:13
Type a message

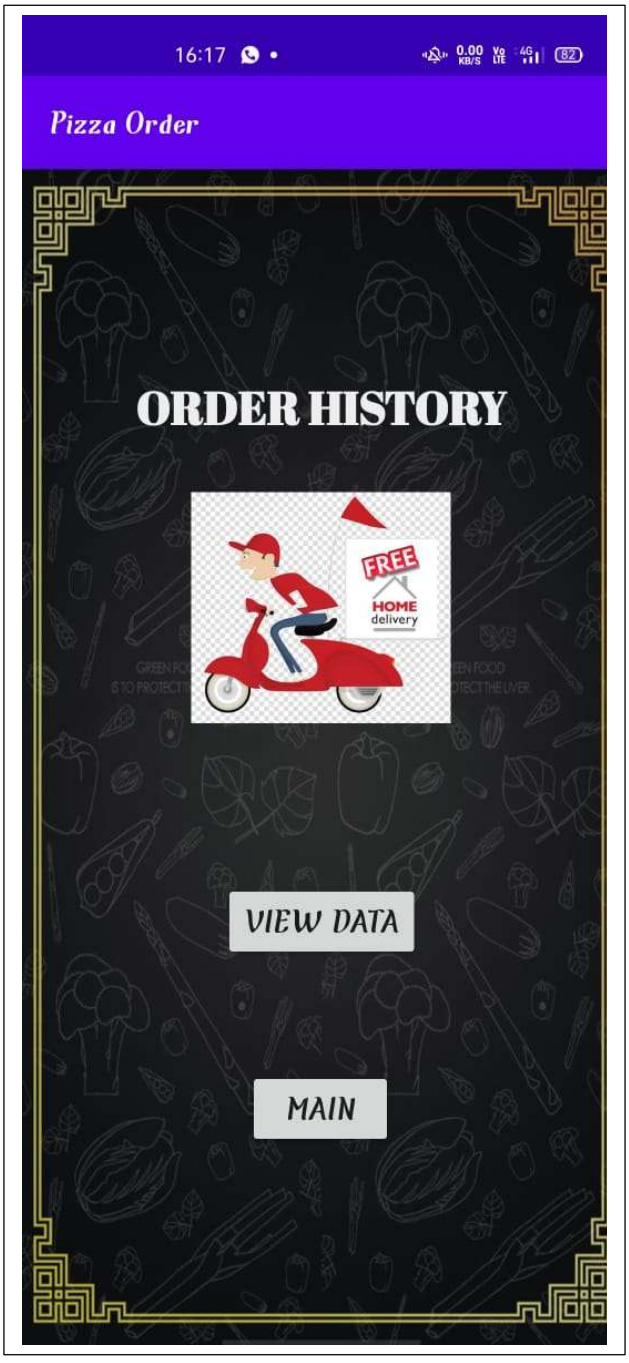

16:15
Pizza Order
Pizza
MENU
ORDER
ABOUT US
ORDER HISTORY


16:15
Pizza Order
Pizza Ordering
MENU
Plzza Chicken Pepperoni
Loaded with extra pepperoni and extra cheese
Hawaiian Chicken
with tomato sauce, beef pepperoni, mozzarella cheese
Aloha Chicken
with thousand island sauce, tuna, pineapples, onions, mozzarella cheese.
BACK


16:16
Pizza Order
Pizza Ordering
ORDER
Pizza Menu
Size:
Small - RM5
Medium -RM10
Large - RM15
Topping:
Seafood - RM8
Chicken - RM5
Meat - RM6
Heyreshsham
perak
019373788
BIGSTOCK
ORDER


16:16
Pizza Order
Pizza Ordering
ORDER
PIZZA
Your Order:
Pizza Size: Small - RM5
Pizza Topping: Chicken - RM5
Heyreshsham
perak
019373788
Total Price: RM :10.0
Data Inserted
BACK
MAIN


16:16
Pizza Order
Pizza Ordering
ABOUT US
Jalan Tok Nasir
TAMAN
PAUH INDAH
TAMAN
UH JAYA
Jalan Pauh Jaya
Jalan Zamrud
Jalan Berlian
CANADIAN PIZZA
Address: 56, Jalan Putra, Chow Kit, 50350 Kuala Lumpur,
TEL:1-300-88-2525
HOURS: 11:00AM TO 11:00PM
BACK


16:17
Pizza Order
ORDER HISTORY
FREE
HOME
delivery
VIEW DATA
MAIN


16:17
Pizza Order
ORDER HISTORY
FREE
Data
Id :1
CustName : Heyreshsham
VIEW DATA
MAIN